

Digital Logic Design

Dr. Muhammad Usman

Professor

Computer Science

UET Mardan

Email: usman@uetmardan.edu.pk

Negative Number Representation

Options

- ☐ Sign-magnitude
- ☐ One's Complement
- ☐ Two's Complement

Sign-magnitude

Use the most significant bit (MSB) to indicate the sign

0: positive, 1: negative

Problem

Representing zeros?

Do not work in computation

+0	000
+1	001
+2	010
+3	011
-3	111
-2	110
-1	101
-0	100

$$A: \{a_{N-1}, a_{N-2}, \dots, a_1, a_0\}$$

$$A = (-1)^{a_{N-1}} \sum_{i=0}^{N-2} a_i 2^i$$

Example:

$$\begin{aligned} \mathbf{1101}_2 &= (-1)^{\mathbf{1}} \times (\mathbf{1} \times 2^2 + \mathbf{0} \times 2^1 + \mathbf{1} \times 2^0) \\ &= -1 \times (\mathbf{4} + \mathbf{0} + \mathbf{1}) \\ &= -5 \end{aligned}$$

- Problems:

- Addition doesn't work, for example $-6 + 6$:

$$\begin{array}{r} 1110 \\ + 0110 \\ \hline 10100 \text{ (wrong!)} \end{array}$$

- Two representations of 0 (± 0):

1000
0000

One's Complement

Complement (flip) each bit in a binary number
Problem

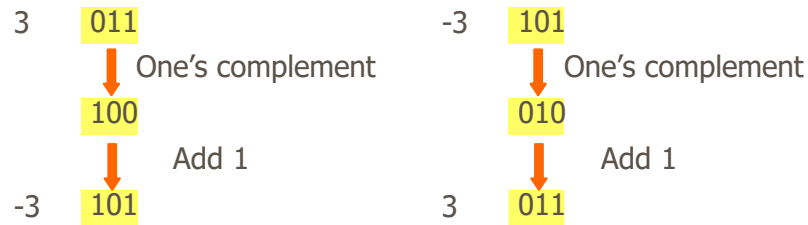
Representing zeros? Do not always work in
computation

Ex: $111 + 001 = 000 \rightarrow$ Incorrect !

+0	000
+1	001
+2	010
+3	011
-3	100
-2	101
-1	110
0	111

Two's Complement

- ❑ Complement(flip) each bit in a binary number and add 1, with overflow ignored
- ❑ Work in computation perfectly



Two's Complement Numbers

$$A: \{a_{N-1}, a_{N-2}, \dots, a_1, a_0\}$$

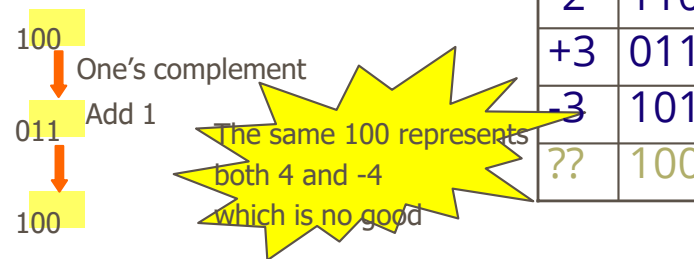
$$A = a_{N-1}(-2^{N-1}) + \sum_{i=0}^{N-2} a_i 2^i$$

Example:

$$\begin{aligned} 1101_2 &= 1 \times (-2^3) + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= -8 + 4 + 0 + 1 \\ &= -3 \end{aligned}$$

Two's Complement

- ❑ Complement(flip) each bit in a binary number and adding 1, with overflow ignored
- ❑ Work in computation perfectly
- ❑ We will use it in this course !



0	000
+1	001
-1	111
+2	010
-2	110
+3	011
-3	101
??	100

Two's Complement

Complement(flip) each bit in a binary number and adding 1, with overflow ignored

Work in computation perfectly

We will use it in this course !

100



One's complement

011



Add 1

100

MSB = 1 for negative
Number, thus 100
represents -4

0	000
+1	001
-1	111
+2	010
-2	110
+3	011
-3	101
-4	100

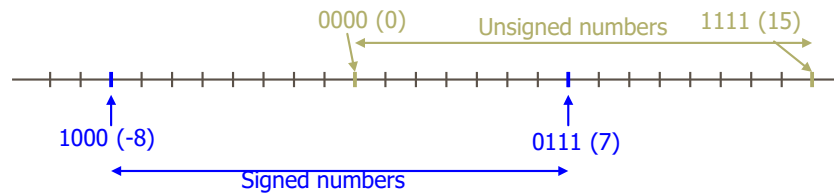
Range of Numbers

An N-bit number

Unsigned: $0 \dots (2^N - 1)$

Signed: $-2^{N-1} \dots (2^{N-1} - 1)$

Example: 4-bit



- **Extend number from N to M bits ($M > N$)**
 - Sign-extension
 - Zero-extension

- Sign bit copied to msb's
- Number value is same
- **Example 1:**
 - 4-bit representation of 3 = 0011
 - 8-bit sign-extended value: 00000011
- **Example 2:**
 - 4-bit representation of -5 = 1011
 - 8-bit sign-extended value: 11111011

- Zeros copied to msb's
- Value changes for negative numbers

- **Example 1:**

- 4-bit value = $0011_2 = 3_{10}$
- 8-bit zero-extended value: **0000**0011 = 3_{10}

- **Example 2:**

- 4-bit value = $1011 = -5_{10}$
- 8-bit zero-extended value: **0000**1011 = 11_{10}

ARITHMETIC OPERATIONS - Binary Arithmetic

- **Single Bit Addition with Carry**
- **Multiple Bit Addition**
- **Single Bit Subtraction with Borrow**
- **Multiple Bit Subtraction**
- **Multiplication**
- **BCD Addition**

Single Bit Binary Addition with Carry

Given two binary digits (X,Y), a carry in (Z) we get the following sum (S) and carry (C):

Carry in (Z) of 0:

Z	0	0	0	0
X	0	0	1	1
+ Y	+ 0	+ 1	+ 0	+ 1
C S	0 0	0 1	0 1	1 0

Carry in (Z) of 1:

Z	1	1	1	1
X	0	0	1	1
+ Y	+ 0	+ 1	+ 0	+ 1
C S	0 1	1 0	1 0	1 1

Multiple Bit Binary Addition

- Extending this to two multiple bit examples:

Carries	<u>0</u>	<u>0</u>
Augend	01100	10110
Addend	<u>+10001</u>	<u>+10111</u>
Sum		

- Note: The 0 is the default Carry-In to the least significant bit.

Single Bit Binary Subtraction with Borrow

- Given two binary digits (X,Y), a borrow in (Z) we get the following difference (S) and borrow (B):

- Borrow in (Z) of 0:

Z	0	0	0	0
X	0	0	1	1
<u>-Y</u>	<u>-0</u>	<u>-1</u>	<u>-0</u>	<u>-1</u>
BS	0 0	1 1	0 1	0 0

- Borrow in (Z) of 1:

Z	1	1	1	1
X	0	0	1	1
<u>-Y</u>	<u>-0</u>	<u>-1</u>	<u>-0</u>	<u>-1</u>
BS	1 1	1 0	0 0	1 1

Multiple Bit Binary Subtraction

- Extending this to two multiple bit examples:

Borrows	<u>0</u>	<u>0</u>
Minuend	10110	10110
Subtrahend	- <u>10010</u>	- <u>10011</u>
Difference		

- Notes: The 0 is a Borrow-In to the least significant bit. If the Subtrahend > the Minuend, interchange and append a – to the result.

Binary Multiplication

The binary multiplication table is simple:

$$0 * 0 = 0 \quad | \quad 1 * 0 = 0 \quad | \quad 0 * 1 = 0 \quad | \quad 1 * 1 = 1$$

Extending multiplication to multiple digits:

Multiplicand	1011
Multiplier	<u>x 101</u>
Partial Products	1011
	0000 -
	<u>1011 - -</u>
Product	110111

Binary Numbers and Binary Coding

- Flexibility of representation
 - Within constraints below, can assign any binary combination (called a code word) to any data
 - as long as data is uniquely encoded.
- Information Types
 - Numeric
 - Must represent range of data needed
 - Very desirable to represent data such that simple, straightforward computation for common arithmetic operations permitted
 - Tight relation to binary numbers
 - e.g. BCD
 - Non-numeric – alpha-numeric
 - Greater flexibility since arithmetic operations not applied.
 - Not tied to binary numbers
 - e.g ASCII, Unicode

Numeric Codes

Decimal digit	8 b_3	4 b_2	2 b_1	1 b_0	8	4	-2	-1	2	4	2	1	Excess-3			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	1	1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	1	0	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	0	1	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	0	0	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	1	1	1	0	1	1	1	0	0	0
6	0	1	1	0	1	0	1	0	1	1	0	0	1	0	0	1
7	0	1	1	1	1	0	0	1	1	1	0	1	1	0	1	0
8	1	0	0	0	1	0	0	0	1	1	1	0	1	0	1	1
9	1	0	0	1	1	1	1	1	1	1	1	1	1	1	0	0

Non-numeric Binary Codes

- Given n binary digits (called bits), a binary code is a mapping from a set of represented elements to a subset of the 2^n binary numbers.
- Example: A binary code for the seven colors of the rainbow
 - Code 100 is not used

Color	Binary Number
Red	000
Orange	001
Yellow	010
Green	011
Blue	101
Indigo	110
Violet	111

Number of Bits Required

- Given M elements to be represented by a binary code, the minimum number of bits, n , needed, satisfies the following relationships:

$$2^n \geq M > 2^{(n-1)}$$

$n = \lceil \log_2 M \rceil$ where $\lceil x \rceil$, called the *ceiling function*, is the integer greater than or equal to x .

- Example: How many bits are required to represent decimal digits with a binary code?

Number of Elements Represented

- Given n digits in radix r , there are r^n distinct elements that can be represented.
- But, you can represent m elements, $m < r^n$
- Examples:
 - You can represent 4 elements in radix $r = 2$ with $n = 2$ digits: (00, 01, 10, 11).
 - You can represent 4 elements in radix $r = 2$ with $n = 4$ digits: (0001, 0010, 0100, 1000).
 - This second code is called a "one hot" code.

DECIMAL CODES - Binary Codes for Decimal Digits

There are over 8,000 ways that you can chose 10 elements from the 16 binary numbers of 4 bits. A few are useful:

Decimal	8,4,2,1	Excess3	8,4,-2,-1	Gray
0	0000	0011	0000	0000
1	0001	0100	0111	0100
2	0010	0101	0110	0101
3	0011	0110	0101	0111
4	0100	0111	0100	0110
5	0101	1000	1011	0010
6	0110	1001	1010	0011
7	0111	1010	1001	0001
8	1000	1011	1000	1001
9	1001	1100	1111	1000

GRAY CODE – Decimal

Decimal	8,4,2,1	Gray
0	0000	0000
1	0001	0100
2	0010	0101
3	0011	0111
4	0100	0110
5	0101	0010
6	0110	0011
7	0111	0001
8	1000	1001
9	1001	1000

- What special property does the Gray code have in relation to adjacent decimal digits?

Binary to Gray Code Conversion

Convention to be used for conversion,

- The most significant bit (left-most) in the Gray code is the same as the corresponding MSB in the binary number.
- Going from left to right, add each adjacent pair of binary code bits to get the next Gray code bit.
Discard carries.

For example, the conversion of the binary number 10110 to Gray code is as follows:

1	−	+	→	0	−	+	→	1	−	+	→	1	−	+	→	0	Binary
↓				↓				↓				↓				↓	
1				1				1				0				1	Gray

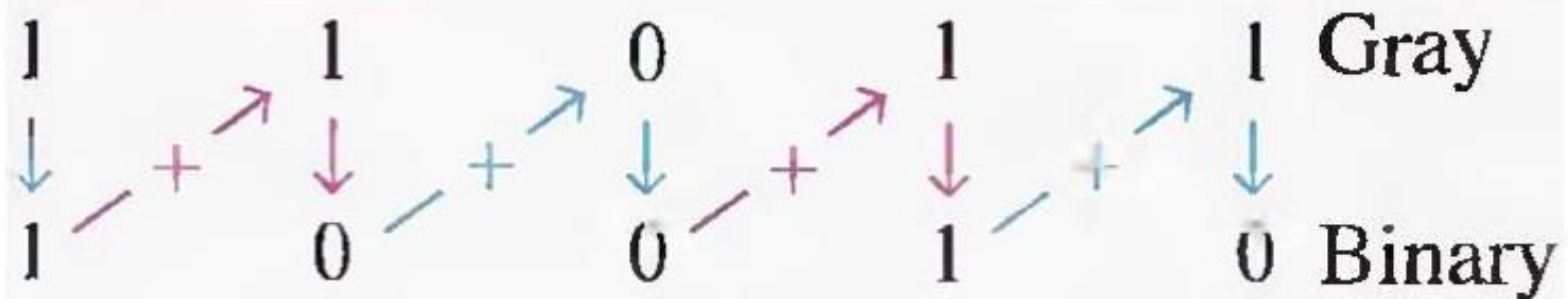
The Gray code is 11101.

Gray Code to Binary Conversion

Convention to be used for conversion,

- The most significant bit (left-most) in the binary code is the same as the corresponding bit in the Gray code.
- Add each binary code bit generated to the Gray code bit in the next adjacent position. Discard carries.

For example, the conversion of the Gray code word 11011 to binary is as follows:



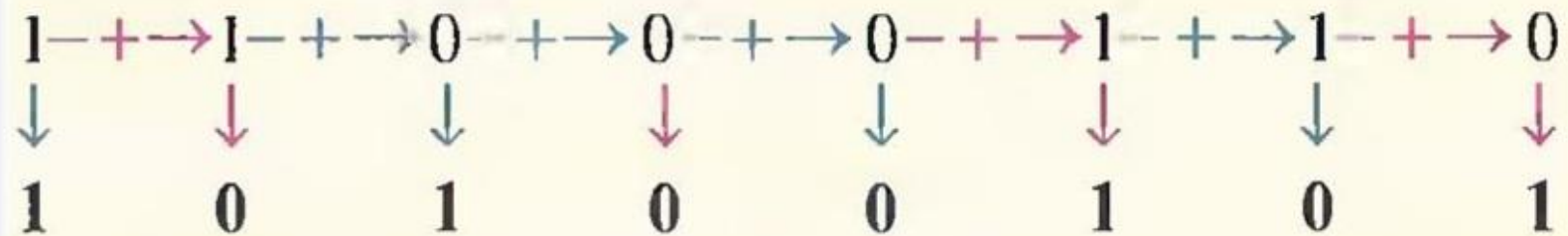
The Binary code is 10010.

Conversion Examples

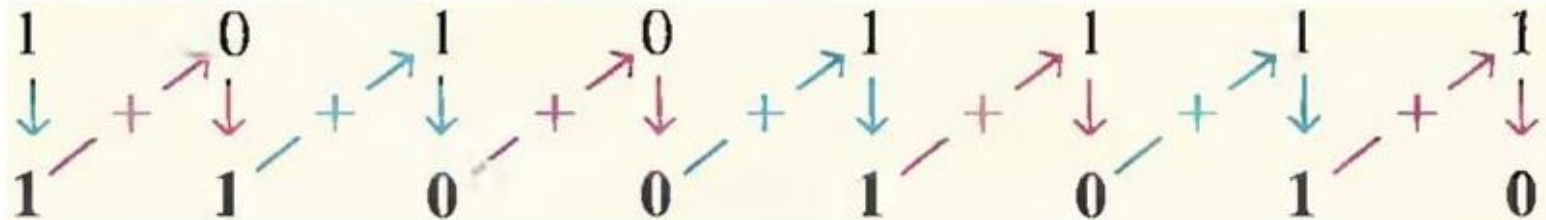
- a) Convert the binary number 11000110 to Gray code.
b) Convert the Gray code 10101111 to binary.

Solution

(a) Binary to Gray code:



(b) Gray code to binary:



Binary Coded Decimal (BCD)

- The BCD code is the 8,4,2,1 code.
- 8, 4, 2, and 1 are weights
- BCD is a *weighted* code
- This code is the simplest, most intuitive binary code for decimal digits and uses the same powers of 2 as a binary number, but only encodes the first ten values from 0 to 9.
- Example: $1001 (9) = 1000 (8) + 0001 (1)$
- How many “invalid” code words are there?
- What are the “invalid” code words?

Warning: Conversion or Coding?

- Do NOT mix up conversion of a decimal number to a binary number with coding a decimal number with a BINARY CODE.
- $13_{10} = 1101_2$ (This is conversion)
- $13 \Leftrightarrow 0001|0011$ (This is coding)

BCD Arithmetic

Given a BCD code, we use binary arithmetic to add the digits:

8	1000	Eight
<u>+5</u>	<u>+0101</u>	Plus 5
13	1101	is 13 (> 9)

Note that the result is **MORE THAN 9**, so must be represented by two digits!

To correct the digit, subtract 10 by adding 6 modulo 16.

8	1000	Eight
<u>+5</u>	<u>+0101</u>	Plus 5
13	1101	is 13 (> 9)
	<u>+0110</u>	so add 6

carry = 1 0011 leaving 3 + cy

0001 | 0011 Final answer (two digits)

If the digit sum is > 9, add one to the next significant digit

BCD Addition Example

- Add 2905_{BCD} to 1897_{BCD} showing carries and digit corrections.

			0
0001	1000	1001	0111
+ 0010	1001	0000	0101
<u> </u>	<u> </u>	<u> </u>	<u> </u>

ALPHANUMERIC CODES - ASCII Character Codes

- American Standard Code for Information Interchange
- This code is a popular code used to represent information sent as character-based data. It uses 7-bits to represent:
 - 94 Graphic printing characters.
 - 34 Non-printing characters
- Some non-printing characters are used for text format (e.g. BS = Backspace, CR = carriage return)
- Other non-printing characters are used for record marking and flow control (e.g. STX and ETX start and end text areas).

ASCII Code

$B_4 B_3 B_2 B_1$	$B_7 B_6 B_5$							
	000	001	010	011	100	101	110	111
0000	NULL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Control Characters

NULL	NULL	DLE	Data link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell	ETB	End of transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete

ASCII Properties

ASCII has some interesting properties:

Digits 0 to 9 span Hexadecimal values 30_{16} to 39_{16} .

Upper case A -Z span 41_{16} to $5A_{16}$.

Lower case a -z span 61_{16} to $7A_{16}$.

- **Lower to upper case translation (and vice versa) occurs by flipping bit 6.**

Delete (DEL) is all bits set, a carryover from when punched paper tape was used to store messages.

Punching all holes in a row erased a mistake!

UNICODE

- **UNICODE extends ASCII to 65,536 universal characters codes**
 - **For encoding characters in world languages**
 - **Available in many modern applications**
 - **2 byte (16-bit) code words**
 - **See Reading Supplement – Unicode on the Companion Website**
<http://www.prenhall.com/mano>

PARITY BIT Error-Detection Codes

- Redundancy (e.g. extra information), in the form of extra bits, can be incorporated into binary code words to detect and correct errors.
- A simple form of redundancy is parity, an extra bit appended onto the code word to make the number of 1's odd or even. Parity can detect all single-bit errors and some multiple-bit errors.
- A code word has even parity if the number of 1's in the code word is even.
- A code word has odd parity if the number of 1's in the code word is odd.

4-Bit Parity Code Example

- Fill in the even and odd parity bits:

Even Parity Message - Parity	Odd Parity Message - Parity
000 _	000 _
001 _	001 _
010 _	010 _
011 _	011 _
100 _	100 _
101 _	101 _
110 _	110 _
111 _	111 _

- The codeword "1111" has even parity and the codeword "1110" has odd parity. Both can be used to represent 3-bit data.